

Machine Learning, diferența dintre o abordare stocastică și una deterministă

Autori: Bonciu Vlad-Matei, Beșliu Stefano

**Academia de Studii Economice București, Facultatea de Cibernetică, Statistică și Informatică
Economică**

Prezenta lucrare își propune să parcurgă și aprofundeze, cu deosebită atenție, noțiuni relevante contextului actual al dezvoltării și inovării diferiților algoritmi, care stau la baza unora dintre cele mai importante și revoluționare ramuri ale informaticii și matematicii aplicate. Învățarea automată (engl. Machine Learning) este un subdomeniu al informaticii, care propune automatizarea și optimizarea mai multor procese, prin implementarea unor algoritmi și metode, menite să „educe” calculatoarele, sau chiar să dezvolte un mecanism capabil să recunoască, deosebească și clasifice un obiect. Astfel, Învățarea Automată prezintă o pondere considerabilă, în ceea ce privește importanța și potențialul de dezvoltare al tehnologiei și algoritmilor moderni, prin crearea unei noi paradigme și atitudini (ex. Deep Learning) față de capacitățile și potențialul unui calculator, ca mai mult decât un instrument de calcul și procesare. Prin evidențierea particularităților de construcție ale conceptelor aferente Algebrei Liniare, Statisticii Descriptive și Computaționale, cât și al Calculului Diferențial, implicate în conceperea unui simplu program de recunoaștere al unor cifre desenate de un utilizator cu ajutorul mouse-ului, dorința autorilor este de a dezvălui o mică parte din vasta știință aflată în continuă formare. Întreaga idee a articolului se bazează pe prezentarea și compararea a două metode de recunoaștere a unei cifre introduse de utilizator, cu scopul de parcurgere în detaliu caracteristicile lor definitorii. În prima parte vom prezenta Rețelele Neurale și proprietățile lor, iar în cea de-a doua K-Nearest Neighbours.

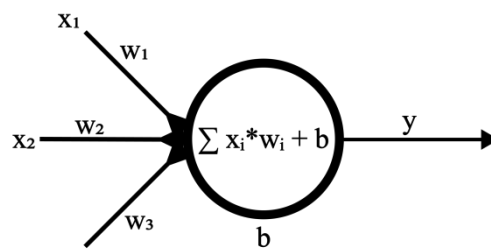
The following paper will cover with undivided attention, notions related to the current development and innovation of the different algorithms that define some of the most important and revolutionary branches of informatics and applied mathematics. Machine Learning, as an informatics subdomain, automatizes and optimizes multiple processes by implementing algorithms and procedures meant to „educate” computers, or even develop a mechanism capable of recognising, differentiating and classifying an object. Thus, Machine Learning presents a new paradigm (Deep Learning), as it drives technological and algorithmic development forward, with the intent to change the individual's attitude towards the abilities that a modern system possesses. By putting the stress on the intricacies of the concepts related to Linear Algebra, Descriptive Statistics/Computational Statistics as well as Calculus, the authors wish to reveal a minuscule part of the ever expanding science that lies behind recognising an integer written by the user, via a mouse. The entire idea of the article is founded on the intent to present and compare two different methods that make it possible for a computer to recognise hand-written digits. The former part will cover Neural Networks and their properties and the latter K-nearest Neighbours.

Cuvinte Cheie: Stocastic, Determinist, Funcții Multivariate, Distanță Euclidiană.

1. Introducere în Învățarea Automată

Învățarea Automată sau Machine Learning este un subdomeniu al informaticii, care vizează totalitatea metodelor, algoritmilor, tehnicilor și paradigmelor de dezvoltare și programare a calculatoarelor în vederea înzestrării acestora cu capacități de recunoaștere, clasificare și previzionare a unor obiecte, evenimente sau aspecte ale realității. Acest proces de „învățare” poate fi destul de ambiguu la început întrucât această noțiune este adesea asociată cu percepția și conștiința umană, aflată în continuă dezvoltare, care în fiecare etapă a vieții își îmbogățește tezaurul de cunoștințe prin experiență și acțiune directă. Machine Learning-ul așadar, își propune același lucru, acela de transpune sau imita procesul de învățare și asimilare desfășurat de om pentru a optimiza și forma o modalitate prin intermediul căreia calculatoarele să poată analiza o problemă la prima vedere și să o caracterizeze bazându-se pe tezaurul de cunoștințe acumulat din experiența precedentă. Acest lucru, bineînțeles că presupune o serie de pași logici, desfășurați algoritmic și

sistematic. Pentru a extinde analogia, în cazul sistemelor de calcul, învățarea se bazează pe largi seturi de date, pe care acestea le asimilează. Acest proces se numește „training” (engl. antrenament), având drept scop acumularea de informații pentru deosebirea și determinarea unor obiecte dintr-o mulțime de obiecte diferite. Pe baza acestui proces se va desfășura și „testing” (engl. testarea), în care sunt încărcate noi seturi de date, pentru a testa acuratețea și corectitudinea modelului. Noțiunea de model, în acest caz, presupune un program/sistem de calcul, care a suferit o serie de schimbări în urma procesului de antrenare și care acum este capabil să ofere o evaluare a unui nou input primit. Această paradigmă, așadar, are la bază un sistem elaborat de tip input-output, în care calitatea output-urilor depinde de volumul și calitatea input-urilor (date corecte și abundente, oglindesc mai bine realitatea cu care se confruntă modelul). Metodele, caracteristicile lor și capacitățile acestora, va fi prezentată ulterior.



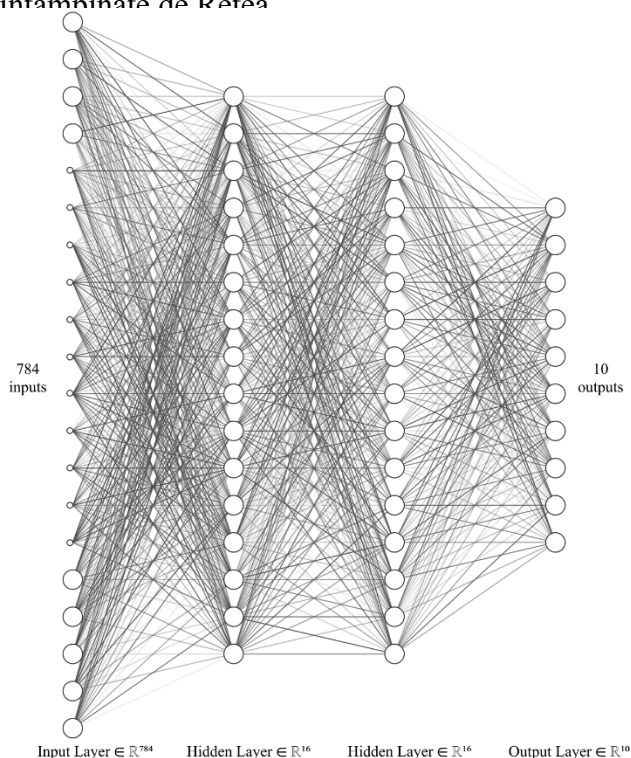
Termenul de Machine Learning, a fost folosit încă din 1950, de către Arthur Samuel, un angajat IBM, care în 1952 a și patentat termenul în urma dezvoltării unui program capabil să cuantifice probabilitatea unui jucător de dame de a câștiga, bazându-se pe poziția fiecărei piese pe tablă. Acesta a antrenat modelul dezvoltat prin expunerea acestuia la mai multe combinații și posibile așezări pe tablă. Concepția de Machine Learning se bazează pe concepte specifice psihologiei umane. Încă din 1949, Donald Hebb, în lucrarea sa intitulată „The Organization of behaviour” evidențiază caracteristicile esențiale dezvoltării ulterioare a teoriei Rețelelor Neurale. Creierul uman este caracterizat printr-o multitudine de legături neuronale, în care fiecare neuron comunică cu ceilalți pentru a transmite informații și impulsuri electrice. Anatomia neuronului, din dendrite, nucleu și axon, vor fi o sursă de inspirație pentru dezvoltarea metodelor de transmitere și procesare a informației pentru Rețelele Neurale, unde Neuronii artificiali funcționează pe baza principiului de input-output, similar cu o funcție. Atomicitatea neuronilor este de asemenea regăsită, astfel formându-se foarte ușor o rețea bogată și complexă, unde fiecare legătură are un anumit rol și pondere.

Machine Learning-ul ulterior este explorat de mai mulți cercetători, precum Frank Rosenblatt, care în 1959 inventează un program și un sistem de calcul numit „Mark 1 perceptron”, în vederea recunoașterii unor imagini/tipare vizuale, dar având dificultăți în procesarea fețelor. În 1967 un alt algoritm de recunoaștere a tiparelor e descoperit de Marcel Pelillo, rezolvând problema celei mai eficiente rute prin cartografierea rutelor de către program. În 1970, Rețelele Neurale deja capătă noi metode de folosire și dezvoltare, prin folosirea unor noi concepte precum „Feed-forward neural networks” (rețele neuronale care au un singur sens, neformând cicluri) și „backpropagation” (folosirea output-ului eronat, pentru corectarea și ajustarea ponderilor legăturilor, astfel adaptând rețeaua la noi scenarii/entități). De asemenea, apar rețelele cu „hidden layers” (engl. straturi ascunse), care au rol intermediar de a procesa și a transmite datele, ajutând la o mai bună interpretare a lor. Din anii 1980, dezvoltarea Machine Learning-ului încetinește, fiind limitat de capacitățile computaționale ale sistemelor de calcul. În 1990, însă, domeniul înflorește cu o serie de descoperiri revoluționare, care vor marca un nou început. Din anii 2000, deja se dezvoltă programe capabile să înțeleagă vocea utilizatorului și să recunoască fețe cu o acuratețe similară cu cea a omului.

Astăzi, Machine Learning-ul este declarat ca fiind „Știința care face ca sistemele de calcul să acționeze, fără a fi programate în mod explicit” (Stanford University). Creșterea lor revoluționară a dus la numeroase noi nișe, precum Analiza Datelor, Publicitate, Explorare Spațiului, Procesarea limbajului și Detectarea tiparelor de fraudă.

2. Rețele Neurale

În cei mai simpli termeni, Rețelele Neuronale pot fi interpretate drept transpunerea neuronilor biologici în ceea ce am putea denumi Neuronii Artificiali. Aceștia, precum neuronii biologici sunt capabili să proceseze și să transmită informații către alți neuroni. Numărul lor mare determină capacitatea acestora de a împărți orice obiect sau problemă în părți foarte mici, astfel înlesnind procesul de învățare și explorare al unui nou scenariu cu care aceștia se întâlnesc. Această perspectivă, însă, este utilă doar la nivel conceptual sau teoretic. În practică, o rețea neurală este reprezentată sub forma unui Graf (V, E) k-partit sau multipartit ponderat, unde V este mulțimea tuturor neuronilor, iar E este mulțimea legăturilor neurale. Fiecare submulțime a grafului k-partit poate fi interpretată drept un „strat” al acelei rețele neurale, iar legăturile dintre fiecare neuron al unui strat, cu neuronii stratului adiacent, sunt numite muchii. Fiecare neuron aferent unui strat este sursa a n muchii, unde n este numărul de neuroni din stratul următor. Aceste muchii, la rândul lor, au o anumită pondere relevantă în procesul de clasificare și deosebire a obiectelor sau elementelor întâmpinate de Rețea



Principala caracteristică definitorie a acestei rețele este aceea că fiecare neuron este considerat o funcție care are drept variabile rezultatele tuturor neuronilor din stratul precedent, iar coeficienții acestor variabile sunt ponderile asociate muchiilor. Acest lucru face posibilă reprezentarea unui neuron artificial prin intermediul legăturilor și valorilor primite de la ceilalți neuroni, sub forma unei combinații liniare:

$$b_i = w_{1i} * a_1 + w_{2i} * a_2 + \dots + w_{ni} * a_n \quad (1)$$

Unde a_i este valoarea neuronului i , iar w_{ji} ponderea muchiei j aferentă neuronului i .

Aceasta poate fi de asemenea reprezentată sub forma unui produs scalar dintre vectorul ponderilor w și vectorul valorilor neuronilor a din stratul anterior:

$$\langle w, a \rangle \quad (2)$$

Fig. SEQ Fig. * ARABIC 2 Rețea Neurală prezentată sub forma unui graf 4-partit

Pentru a ușura procesul de evaluare a valorilor reprezentate de toți acești neuroni artificiali, fiecare dintre aceștia vor fi trecuți printr-o funcție sigmoidă, mai specific funcția Logistică Sigmoidă, astfel, combinația liniară prezentată anterior va avea rol de argument al Funcției Logistice pentru neuronul curent:

$$f: \mathbb{R} \Rightarrow [0;1] \quad f(x) = \sigma(x) = 1 / (1 + e^{-x}) \quad (3)$$

Am ales această funcție deoarece valorile sale se încadrează în intervalul $[0 ; 1]$. Această proprietate este importantă deoarece facilitează procesul de determinare al valorilor primite de către neuronii straturilor următoare și al optimizării și corectării ponderilor muchiilor. Prin încadrarea valorilor în aceste intervale, am făcut posibilă restrângerea problemei la determinarea magnitudinii fiecărui neuron în parte, de la o valoare nulă la o valoare pozitivă, ușor de utilizat și transferat.

Odată determinate aceste noțiuni, întreaga problemă se restrânge la rolul și importanța fiecărui neuron în parte. În cazul în care dorim să creăm o rețea neurală, capabilă să recunoască un număr scris cu mouse-ul de către utilizator, pe un plan de desfășurat pe 28*28 pixeli, rețeaua neurală trebuie să învețe să identifice fiecare dintre cele 10 cifre posibile (0–9). Pentru un asemenea scenariu, rețeaua are nevoie să divizeze fiecare număr, în părți mici, care la rândul lor, să fie puse laolaltă, pentru a forma tipare și forme recunoscute. În acest caz, trainingul acestei rețele se va desfășura pe mai multe straturi, unde fiecare va analiza un aspect anume al cifrei înscrise de utilizator. De exemplu, la nivel logic, primul strat al rețelei poate ține cont de numărul și locația pixelilor colorați de utilizator, pentru a filtra cazurile posibile de cifre. Prin acest prim test, în funcție de intensitatea și numărul pixelilor, fiecare neuron din strat va reprezenta o valoare între 0 (absența) și 1 (prezența).

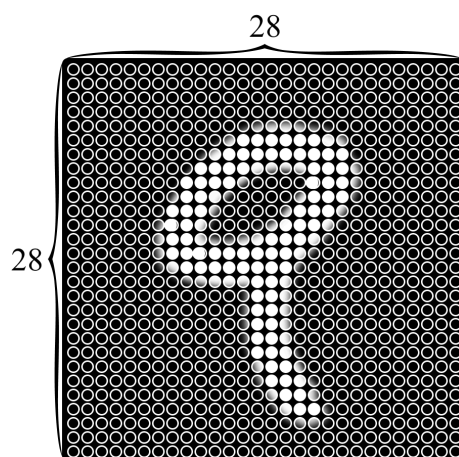


Fig. SEQ Fig. * ARABIC 3
Reprezentarea planului de 784 de pixeli

Acești pixeli, conceptual vor fi evaluați în grupuri, pentru a putea reduce numărul de neuroni (fiecare neuron stochează o valoare aferentă unui rând sau părți anume din plan). Un al doilea strat ar avea apoi rolul de a identifica grupuri de asemenea pixeli, care formează forme mai complexe, cum ar fi semicercuri, cozi, capete sau chiar să se folosească de absența unor caracteristici într-o parte a planului, pentru a micșora domeniul de căutare/determinare. Acest proces se desfășoară pe un număr destul de mare de straturi, până când rețeaua are destule date pentru a determina un rezultat.

Totuși, în cazul Rețelei propriu-zise, identificarea tiparelor și a formelor se desfășoară prin intermediul unor șabloane greu de recunoscut și perceput de om. Acestea percep întreaga cifră ca pe un nor de puncte.

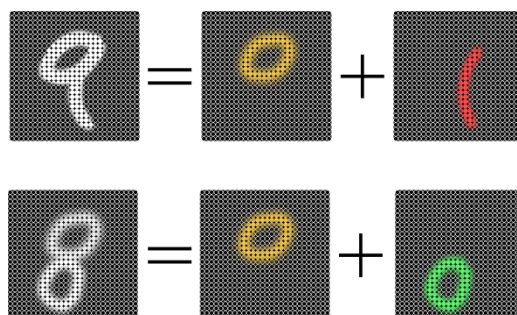
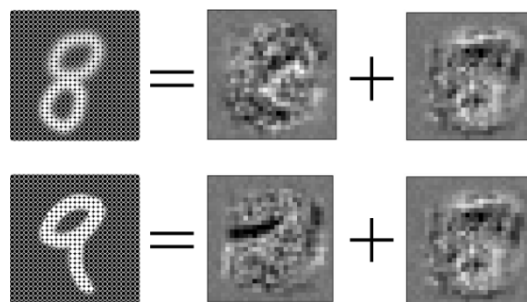


Fig. SEQ Fig. * ARABIC 4 Exemplu de metodă de împărțire a cifrelor, în părți și forme, din perspectiva umană



Stratul terminal va include 10 neuroni. Fiecare ținând cont de certitudinea rețelei raportate la posibilitatea ca cifra scrisă de utilizator să fie cea reprezentată de neuron. De exemplu, în cazul în care utilizatorul decide să deseneze cifra 9, rețeaua neurală, în urma clasificării și evaluării cifrei va compara valorile fiecărui neuron, pentru a determina cifra desenată. În acest caz, prezența unui cerc superior va duce la valorile diferite: neuronii 1, 4, 5, 7 vor avea valori foarte mici, aproape de 0, în timp ce neuronii 2, 3, 6, 8, 9, 0 vor avea valori pozitive sau aproape de 1 (foarte mari în cazul lui 8, 9 și 0). Calitatea evaluării finale depinde de numărul de antrenamente și calitatea datelor folosite.

3. Backpropagation: Algoritmul stocastic

Ținând cont de conceptele prezentate, acum putem parcurge algoritmul din spatele întregului proces de antrenare și dezvoltare al rețelei. Am învățat cum funcționează rețeaua, dar nu am clarificat metodele prin care aceasta ajunge să recunoască cifrele introduse.

În practică, rețelele neurale nu sunt capabile de la bun început să determine cifrele corect. Acestea pleacă din punctul în care fiecare pondere a muchiilor rețelei este definită total aleatoriu. Acest lucru este relevant, deoarece încadrează acest algoritm în categoria celor stocastici. Un algoritm stocastic este determinat de natura aleatorie a dezvoltării sale.

În rețelele neurale, ponderile muchiilor fiecărui strat reprezintă componenta inițial aleatorie a algoritmului de Backpropagation. De asemenea, vom introduce pentru fiecare argument al funcțiilor neuronilor, un “bias value” (engl. Valoare de părtinire). Ideea din spatele acestei valori este de a asigura un prag necesar fiecărui neuron, pentru a fi încadrat în zona pozitivă (activată). Introducerea unui bias ajută și la determinarea mai precisă și ușoară a gradului de intensitate al existenței unei legături.

$$f(b_i) = \sigma(w_{1i} * a_1 + w_{2i} * a_2 + \dots + w_{ni} * a_n + p_i) \quad (4)$$

Formula finală a fiecărui neuron, unde p_i este valoarea de părtinire

Prin existența a două variabile care influențează valoarea argumentului este posibilă manipularea acestor valori, în vederea obținerii rezultatelor dorite pentru fiecare scenariu în parte.

Valorile acordate aleatoriu ponderilor și bias-urilor, vor produce, inițial, rezultate eronate. Backpropagation se bazează pe identificarea acestor erori și efectuarea următoarelor operații:

- 1) Folosindu-ne de funcția Softmax, vom normaliza fiecare rezultat al neuronilor terminali. Acest lucru este necesar, pentru a forma o serie de distribuție de probabilități din toate cele 10 rezultate. Astfel, suma celor 10 neuroni va fi întotdeauna 1, proprietate utilă în pasul următor.

$$\sigma_{\text{Softmax}}: \mathbb{R} \Rightarrow (0;1), \quad \sigma_{\text{Softmax}}(z_i) = e^{z_i} / (\sum e^{z_j}) \quad (5)$$

Unde suma include toate cele 10 valori ale stratului terminal, iar e^{z_i} este al i-lea termen din strat.

- 2) Pe baza acestei normalizări, vom calcula funcția de cost, determinată de Cross Entropy function (engl. Entropie încrucișată), o funcție aferentă Teoriei Transmiterii Informației.

$$C_{CE} = -1/N * \sum (y_i * \ln(n_i) - (1 - y_i) * \ln(1 - n_i)) \quad (6)$$

Unde y_i este probabilitatea dorită, în timp ce n_i este probabilitatea obținută în urma normalizării.

- 3) În funcție de rezultatele obținute, vom putea monitoriza calitatea evaluării. Un număr apropiat de 0 indică o acuratețe ridicată, în timp ce unul îndepărtat, determină necesitatea îmbunătățirii rețelei. Bineînțeles, după cum putem vedea, pentru funcția de cost, în cazul în care probabilitățile obținute sunt asemănătoare cu cele dorite, rezultatul funcției va fi apropiat de 0.

Pentru fiecare antrenament în parte, ținem cont de această funcție de cost. Dacă avem 400 de asemenea antrenamente posibile, vom avea 400 de funcții de cost. Pe acestea le însumăm și împărțim la numărul de antrenamente, în vederea obținerii unei medii:

$$C_{\text{MEDIU}} = \sum C_i * 1/n \quad (7)$$

Aceasta este deosebit de utilă, deoarece ne oferă un indicator sintetic, capabil să cuantifice calitatea rețelei neurale. O funcție de cost mare denotă ineficiența rețelei. Calculăm media funcțiilor de cost, deoarece dorim să aflăm un coeficient reprezentativ pentru toate posibilele teste. Dacă am corecta ponderile în funcție de un singur scenariu posibil, bineînțeles că rețeaua noastră ar recunoaște orice cifră ca fiind acea cifră pentru care s-a antrenat.

Un alt motiv pentru care calculăm funcția de cost este de a afla efectul fiecărei muchii ponderate și valoare de părtinire în rețea. Funcția de cost acceptă drept argument un vector format din toate ponderile și valorile de părtinire din rețea, mai precis un vector k -dimensional, unde k este numărul total de muchii ponderate și valori de părtinire. Ne interesează acest vector, întrucât ne va oferi soluția spre minimizarea funcției de cost.

$$W = [w_1 \ w_2 \ w_3 \dots w_n \ p_1 \ p_2 \dots p_t] \quad (8)$$

Unde n este numărul total de ponderi și t numărul total de bias-uri (din definiție, $t + n = k$)

De aceea, funcția aceasta reprezintă punctul de start al algoritmului. Elementele vectorului oferit drept argument sunt responsabile pentru rezultatul obținut din funcția de cost. Astfel, determinarea aportului fiecărei ponderi și bias din rețea este cheia către îmbunătățirea acesteia. Derivarea funcției de cost, în funcție de fiecare variabilă, va oferi o perspectivă asupra posibilităților de minimizare. În plus, vom cunoaște și importanța modificării fiecărei variabile (pentru un caz, anumite ponderi au mai multă importanță, pentru altele, aceleași ponderi sunt nesemnificative în rezultatul final).

Vom introduce noțiunea de Gradient al funcției de cost, adică un vector k -dimensional, unde fiecare element reprezintă derivata parțială a funcției, în raport cu o pondere sau bias. Din definiție, cunoaștem faptul că pentru o funcție multivariabilă, Gradientul determină direcția în plan a funcției pentru cea mai mare rată de creștere. Mai precis, fiecare element din vector constituie o valoare relevantă în contextul maximizării funcției de cost.

$$\nabla C_{\text{MEDIU}} = [\partial C_{\text{MEDIU}}/\partial w_1 \dots \partial C_{\text{MEDIU}}/\partial w_n \ \partial C_{\text{MEDIU}}/\partial p_1 \dots \partial C_{\text{MEDIU}}/\partial p_t] \quad (9)$$

Unde $\partial C_{\text{MEDIU}}/\partial w_i$ este derivata parțială a C_{MEDIU} , în raport cu variabila w_i , analog pentru $\partial C_{\text{MEDIU}}/\partial p_i$.

Intuitiv, ceea ce vom face este să scădem din funcția de cost, gradientul acesteia, în funcție de o valoare α , numită rată de antrenare. Aceasta, determină mărimea „pașilor” către minimizarea funcției. Introducem acest α , deoarece dorim ca schimbarea funcției de cost să se desfășoare treptat, astfel încât să putem reevalua funcția regulat. Acest proces, poartă numele de Coborâre prin intermediul Gradientului, pentru că după fiecare scădere, funcția de cost converge către o valoare minimă locală, prin folosirea celei mai scurte și abrupte căi.

$$W_{t+1} = W_t - \alpha \nabla C_{\text{MEDIU}} \quad (10)$$

Unde W_{t+1} este argumentul funcției după scăderea gradientului, W_t este argumentul inițial, ∇C_{MEDIU} este Gradientul funcției noastre, iar α rata de antrenare.

Numele de Backpropagation vine de la ideea algoritmului de a pleca de la funcția de cost, pe care printr-o suită de pași o minimizăm. Procesul este amplu și laborios din punct de vedere computațional, deoarece este parcursă întreaga rețea de numeroase ori. De asemenea, pentru calculul derivatelor parțiale, trebuie să ne folosim de Regula derivării funcțiilor compuse, care presupune derivarea în lanț a tuturor funcțiilor din spatele funcției principale (natura intrinsecă a rețelei, caracterizată prin interdependențe numeroase și complexe, determină necesitatea pătrunderii în adâncime, pentru aflarea tuturor derivatelor care influențează și determină derivata coeficientului final).

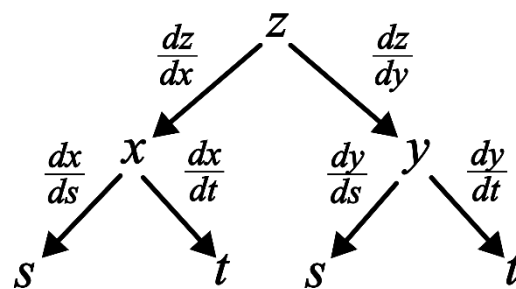
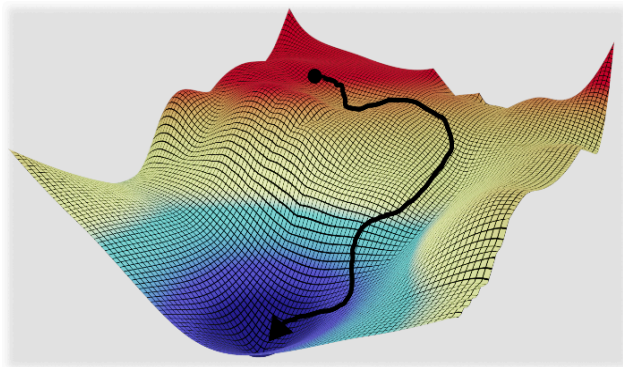


Fig. 5 Exemplu de aplicare a Reguli Derivatelor Compuse

În practică, pentru rețele de mărimi foarte mari, această metodă este foarte solicitantă și ineficientă din punct de vedere computațional. Pentru asemenea situații, este folosită o abordare numită Stochastic Gradient Descent (eng. Coborârea Stocastică prin intermediul Gradientului), care presupune efectuarea antrenamentului rețelei, pe submulțimi de antrenamente. De exemplu, dacă avem 60000 de posibile input-uri pe care le putem oferi rețelei în vederea antrenării acesteia, încărcarea lor într-un singur pas, ar fi foarte solicitantă și laborioasă pentru Sistemul de Calcul. Din punct de vedere computațional, este ineficient să efectuăm toate antrenamentele deodată, deși rezultatele ar fi precise și relevante. În acest caz, putem aborda problema prin organizarea aleatorie a input-urilor, pe submulțimi de 500, 1000 de antrenamente. Acest lucru va afecta rata de minimizare a funcției de cost, dar nu și destinația acesteia. Organizarea pe submulțimi, determină o coborâre pe o pantă mai puțin abruptă, dar care conduce tot la un rezultat bun. Principala diferență este dată de punctul de plecare al algoritmului, unde în cazul celui stocastic, începutul este unul nedeterminat, dar care pe parcurs, se aliniază cu direcția impusă de Gradientul global al funcției de cost.



Această abordare, permite astfel îmbunătățirea rețelei cu o precizie mai mică inițial, dar cu o suprasarcină redusă, ceea ce pentru dezvoltatorii de modele ample și complexe, este esențial în obținerea unor rezultate bune în limita capacităților sistemului.

După cum se observă, acest algoritm ne oferă o definiție a Învățării Automate, ca un proces complex bazat pe multe calcule și formule matematice implicate. În ciuda acestui aspect, acesta doar dezvăluie o nouă perspectivă asupra

nt, într-un

învățării, unde chiar și un sistem de calcul efectuează pași similari cu cei ai unui om. De la întâlnirea unei noi probleme, la eșuarea determinării soluției, până la reparcurgerea problemei cunoscând rezultatul dorit, Machine Learning-ul nu face nimic altceva decât să transpună, în manieră algoritmică, metoda pe care orice individ o folosește de la începutul vieții pentru a învăța și adapta la mediul aflat în continuă schimbare. Analog cu modul în care un individ învață să meargă pe bicicletă prin numeroase încercări și metode, calculatorul percepe întreaga problemă ca pe o funcție de cost, pe care trebuie să o minimizeze, pentru a-și atinge obiectivul, astfel reiterând prin ceea ce a întâmpinat, cu speranța de a-și îmbunătăți abordarea.

Totuși, o caracteristică a acestor algoritmi, este faptul că Antrenamentul Modelului, se axează strict pe un singur aspect. Acest lucru, sugerează faptul că în ciuda capacităților sale de

recunoaștere a cifrelor, acesta nu poate să le reproducă sau transcrie la rândul său. Metoda noastră, este una rudimentară și specifică, bazată pe un anumit context, cu variații în care cifra nu e privită ca o entitate de sine stătătoare, ci ca pe o serie de tipare și numere folosite în calculul unei funcții. Astfel, calculatorul nu poate să facă, în cazul acestui tip de algoritm, asocieri și inferențe, referitoare la alte posibile acțiuni sau contexte.

4. K-nearest Neighbours: Algoritmul determinist

Până acum, am discutat despre un tip de algoritm al Machine Learning-ului, bazat pe un model stocastic de inițializare al optimizării și dezvoltării sale. Algoritmul pe care îl vom discuta acum, este mai simplu de înțeles și procesat din punct de vedere computațional, folosindu-se de anumite concepte aferente Statisticii și Algebrei Liniare.

K-Nearest Neighbours, este un algoritm determinist, folosit foarte des în analiza și clasificarea datelor procesate de sistemele de calcul, mulțumită complexității reduse, implementării ușoare și sarcinii computaționale mici în raport cu alți algoritmi de clasificare.

Acesta este un algoritm de clasificare, deoarece oferă posibilitatea determinării categoriei unui obiect, prin consultarea unei baze de date organizate pe clase. Aceste clase sunt determinate la începutul procesului de antrenare al modelului, fiind recomandată folosirea unei baze vaste și cu cât mai puține cazuri aberante, pentru a optimiza funcționalitatea. De asemenea, toate aceste date deja cunoscute vor fi transpuse sub forma unor vectori, ai căror elemente reprezintă valorile anumitor caracteristici esențiale în catalogarea și determinarea claselor. Am ales vectorii, deoarece aceștia pot fi amplasați într-un plan n-dimensional, cu o reprezentare sub formă de Clustere/Nori (obiecte/cazuri aparținente aceleiași clase, vor avea valori similare și implicit poziție similară în plan).



Litera K, din numele algoritmului, face referire la numărul de vecini pe care utilizatorul îi poate impune acestuia, pentru aflarea clasei obiectului. Acest lucru, este relevant, deoarece calitatea datelor și tendința acestora, poate fi un factor determinant în alegerea numărului K. În cazul unor baze de date cu valori aberante, este de preferat ca numărul să fie mai mare, pentru a evita rezultate eronate și estimări tendențioase. De asemenea, K-ul, este în general ales drept un număr impar, cu scopul de a evita scenariile în care numărul de obiecte aparținente clasei A, să fie același cu cele al clasei B. În cazul existenței unor vecini din clase de mai multe tipuri (de ex, pentru k = 5, să avem 1 obiect din A, 3 din B și 1 din C), alegerea se va face după principiul „Vot majoritar”.

Aflarea vecinilor presupune calcularea unei distanțe. Având în vedere faptul că vom lucra cu vectori de date de dimensiuni destul de mari, acest lucru presupune determinarea distanței prin două formule. Prima este cea a distanței Euclidiene în plan, al cărei formulă este următoarea:

$$d(x, y) = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]} \quad (11)$$

Unde x și y sunt vectori n-dimensional sau sub forma de normă euclidiană a diferenței vectorilor:

$$\|x - y\| \quad (12)$$

A doua formulă este cea a distanței Manhattan:

$$d(x, y) = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n| \quad (13)$$

Noi vom utiliza formula distanței euclidiene, deoarece lucrăm cu date continue și aceasta este mult mai sensibilă la diferențe de magnitudine.

Procesul de învățare în cazul acesta, este mult mai simplu și ușor de implementat. Determinarea vectorilor cifrelor scrise de utilizator, se desfășoară prin simularea turnării unei găleți cu apă peste acestea. Mai exact, algoritmul va turna apă pe numărul nostru de 5 ori. O dată pentru fiecare dintre cele patru direcții cardinale (Nord, Sud, Est, Vest), iar încă una pentru toate acele direcții, laolaltă (similar cu scufundarea acestuia într-o găleată cu apă). Apoi, acesta va împărți cifra respectivă în două, prin trasarea unei drepte verticale (dreapta este verticală din perspectiva direcției din care este turnată apa, nu a utilizatorului), pentru toate direcțiile cardinale.

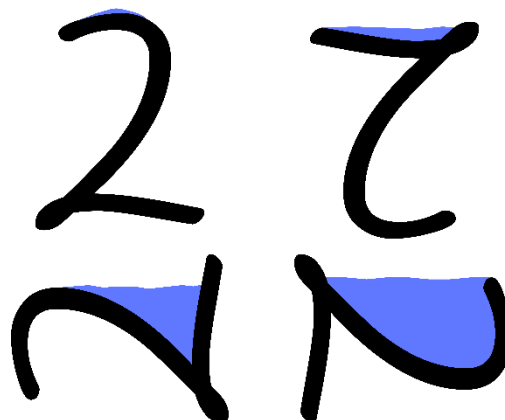


Fig.8 Reprezentare intuitivă a procesului de determinare

Împărțim cifra în două, deoarece putem să le deosebim de altele mai ușor, în momentul în care are loc căutarea vecinilor. Mai precis, prin creșterea dimensiunii vectorului nostru (de la 5 elemente la 9), adăugăm un strat extra de complexitate, care va ține cont de zone mai mici ale planului și de particularități care ar fi fost neobservate într-o analiză mai simplă.

În ceea ce privește calcularea cantității de apă rămasă pe cifră, aceasta are loc prin interpretarea planului în care se află numărul, drept o matrice. Practic, planul de 784 de pixeli este o matrice care ține cont de numărul de pixeli colorați cu albastru și de numărul de pixeli colorați cu alb (fundalul) sau negru (cifra respectivă). Astfel, formula de calcul este următoarea:

$$\text{Număr Pixeli Albaștri} / (\text{Număr Total de Pixeli} / 2) \quad (14)$$

De observat că formula e valabilă pentru cele 4 direcții cardinale, unde cantitatea de apă rămasă pe cifră este evaluată pe cele două jumătăți.

Pentru scenariul 5, unde întregul număr e înconjurat de apă, avem:

$$\text{Număr Pixeli Albaștri} / \text{Număr Total de Pixeli} \quad (15)$$

Așa cum am menționat, pentru toate aceste teste, înregistrăm câte un element al vectorului aferent cifrei analizate. Pentru programul nostru, vectorul va fi 9-dimensional (2 elemente pentru fiecare direcție cardinală, plus unul pentru scufundarea cifrei). Vectorul nostru va arăta astfel:

$$v = [v_1 \ v_2 \ v_3 \ \dots \ v_8 \ v_9] \quad (16)$$

Fiind un vector apartenent spațiului vectorial R^9 , închipuirea vectorului în plan este dificilă și contraintuitivă. Totuși, așezarea tuturor coeficienților calculați într-un vector, ne oferă oportunitatea să ne organizăm mai ușor fiecare rezultat al antrenamentelor și testelor efectuate.

În final, pentru rezultatele obținute, vom calcula distanța euclidiană, în vederea identificării celor mai apropiați vectori (în funcție de numărul de vectori căutați determinați de K).

Un aspect interesant și relevant, îl reprezintă faptul că acest algoritm este determinist. Determinismul presupune capacitatea unui algoritm de a oferi un rezultat unic și consistent, pentru o situație inițială anume, indiferent de numărul de încercări. În situația noastră, KNN este determinist, deoarece două input-uri identice, vor produce același rezultat. De asemenea, este

considerat un „Lazy Learner” (engl. Învățător Leneș), întrucât acesta preia datele într-un format preprocesat. Practic, întregul proces de învățare se rezumă doar la asimilarea datelor. Acesta e în antiteză cu alte metode precum Rețelele Neuronale, unde are loc o analiză amplă a datelor, combinată cu un algoritm de formare și îmbunătățire al metodei de clasificare și organizare, pe parcursul antrenamentelor.

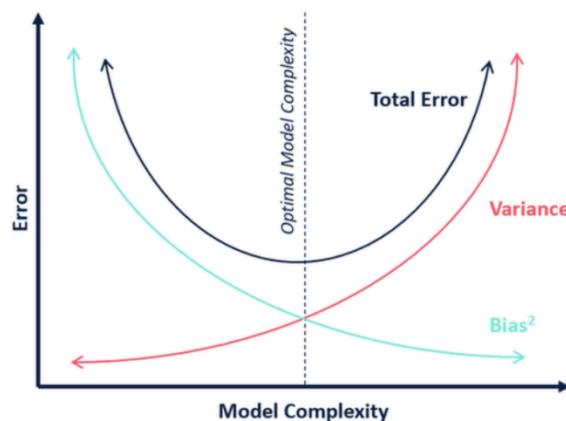
5. Rezultatele algoritmilor

Având algoritmii prezentați, acum putem determina modul în care fiecare dintre aceștia se deosebesc unul de celălalt și în ce scenariu funcționează optim.

Rețelele Neuronale, respectiv Backpropagation, sunt mult mai solicitante din punct de vedere computațional. Acest lucru, presupune existența anumitor limite în ceea ce privește posibilitatea unui calculator/sistem de calcul de a rula un asemenea algoritm. Din experiența autorilor, implementarea Rețelelor Neuronale a fost foarte laborioasă în ceea ce privește gestiunea memoriei și depășirea anumitor constrângeri legate de specificațiile tehnologiei utilizate. Operația de antrenare a modelului a fost sursa principalelor probleme, întrucât procesul presupune parcurgerea unui set de date de 60.000 de posibile scenarii, de mai multe ori. Mai precis, pentru antrenarea unei rețele capabile să evalueze corect o cifră, este nevoie ca modelul să capete experiență prin parcurgerea repetată a antrenamentelor. De asemenea, din cauza naturii procesului de antrenare, modelul tinde să se confrunte cu Randamente Descrescătoare. Acest lucru este foarte des întâlnit în Machine Learning, întrucât majoritatea algoritmilor moderni de dezvoltare a modelelor sunt proiectați în spații vectoriale foarte ample, de dimensiuni mari. Operarea cu asemenea variabile și vectorii, duc la așa-numitul „Curse of Dimensionality” (engl. blestemul dimensionalității), unde seturile de date foarte ample, supun acești algoritmi la probleme de optimizare, în fazele ulterioare de antrenament. Algoritmii de Gradient Descent, este foarte susceptibil unui asemenea fenomen, în fazele intermediare sau finale ale optimizării funcției de cost, unde antrenamentele desfășurate nu mai au un efect atât de semnificativ asupra funcției. În unele cazuri, acest fenomen poate duce chiar la redundanță, deoarece funcția rămâne într-o zonă intermediară, nesatisfăcând cerințele noastre legate de precizie și eficiență.

În ciuda acestor aspecte, Rețelele Neuronale, prin abordarea lor Stocastică, oferă o nouă perspectivă asupra posibilităților de rezolvare a unor probleme. Acestea nu sunt pretențioase în ceea ce privește obiectul antrenamentelor, în sensul că pot oferi o soluție optimă unei suite de probleme. Principala lor proprietate, este capacitatea de adaptare și integrare într-o gamă vastă de scenarii. Acest lucru, este evidențiat de progresul și importanța lor în actualele tehnologii, unde Rețelele Convoluționale sunt capabile de a identifica mai multe elemente simultan, și de a crea șabloane proprii pentru a ușura procesul de clasificare. Versatilitatea, combinată cu capacitatea lor de a se dezvolta pe parcurs, este de o deosebită importanță și în alte tehnologii mai avansate, precum Large Language Models (LLM, modele folosite pentru Generative AI, precum Chat GPT o-4, Deep Seek R1, Gemini etc.) sau Deep Fake (Tot din categoria Gen AI, modele capabile să imite sunete precum vocea unui om sau să creeze videoclipuri cu obiecte sau persoane publice). Astfel, capacitatea lor de procesare a datelor și de autodezvoltare, constituie baza tuturor tehnologiilor și paradigmelor aferente modelelor moderne.

În ceea ce privește acuratețea algoritmului, pe un set de date de 60000 de scenarii/antrenamente posibile, avem o probabilitate de 95% succes în a evalua și clasifica corect o cifră introdusă de utilizator. Pe partea aceasta, putem afirma faptul că Rețelele Neuronale sunt mult



mai pretențioase în ceea ce privește volumul de date necesar, prin prisma faptului că învățarea are loc gradual.

K-nearest neighbours, pe de altă parte, este un algoritm mult mai simplu și limitat. Acesta nu include o arie atât de vastă, dar oferă o soluție simplistă și ușor de implementat. Aplicabilitatea lui e dependentă de scenariul impus și de scop. Acesta este de asemenea susceptibil Randamentelor Descrescătoare, dar dintr-un alt punct de vedere. Pentru seturi mari de date, problema revine din existența a multor valori aberante sau a prea multe variabile care pot afecta magnitudinea distanțelor. În ciuda acestui lucru, problema de mai sus nu este la fel de severă ca în cazul Rețelelor Neurale. Avantajul algoritmului, este dat de faptul că este un Lazy Learner, ceea ce determină ușurința acestuia de a se antrena și dezvolta. Prin preluarea datelor în forma lor originală, antrenamentul se rezumă la procesarea acestora. Astfel, calitate metodei este dată de volumul și acuratețea datelor, nu de procesele desfășurate de model, sau de tehnicile/algoritmii din spate. Abordarea deterministă, este relevantă din perspectiva în care majoritatea algoritmilor de Machine Learning sunt probabilistici sau stocastici. Putem interpreta acest lucru ca fiind atât pozitiv, cât și negativ. Din perspectiva pozitivă, algoritmul oferă rezultate ușor de investigat și analizat, ceea ce ușurează procesul unui dezvoltator de a modifica anumite aspecte ale sale (de exemplu, dacă modelul nostru evaluează corect toate cifrele, înafară de 0 și 8, pe care le confundă, dezvoltatorul poate să ofere mai multe date referitoare la aceste cazuri, pentru a reduce acea marjă de eroare). Natura „statică” a algoritmului, reduce gradul de incertitudine legat de calitatea sa (tot ceea ce este determinat într-un fel, poate fi interpretat ca atare și optimizat printr-o bază de date mai amplă sau reprezentativă). Din cea negativă, acesta este limitat de acest aspect, în comparație cu alți algoritmi, întrucât nu poate fi extins în multe domenii sau scenarii, rezumându-se la o clasificare prin comparație. Astfel, calitatea rezultatelor sale, sunt în totalitate dependente de ceea ce procesează și preia ca atare. Totuși, acest lucru se dovedește a fi un avantaj pentru sistemele de calcul cu specificații reduse, algoritmul având o rată de succes de 80%, în cazul unui set de date de doar 700 de intrări/scenarii.

Link Prezentare Powerpoint:

<https://docs.google.com/presentation/d/1eCR1fQXKzmv4bwdxZD7aKk00gGt5KCXW/edit#slide=id.p1>

6. Concluzii

În urma acestui articol, am avut ocazia să explorăm un domeniu fascinant al informaticii, aflat într-un proces continuu de dezvoltare și reinterpretare. În actualitate, Machine Learning-ul are o importanță mai mare ca niciodată, fiind punctul de plecare al oricărei idei inovatoare și futuristice legate de posibilitatea automatizării și îmbunătățirii societății. Era Inteligenței Artificiale este încă în faza de incipiență, promițând o suită de noi posibilități și metode de a rezolva problemele, pe care mulți experți sunt reticenți în a le accepta.

Prin compararea celor doi algoritmi, am reușit să observăm cum procesul de învățare automată este doar un alt mod de a programa un sistem de calcul. Scenariul în care programatorul este cel care programează metoda prin care modelul se autoprogramează și învață, devine din ce în ce mai real și relevant. Dacă prin metoda stocastică, care integrează Rețelele Neurale, am parcurs modul în care un creier digital învață și reușește să obțină rezultatul dorit, prin metoda deterministă, am schimbat doar abordarea și punctul de plecare, păstrând același obiectiv. Prin aceasta, Machine Learning-ul devine din ce în ce mai aproape de metodele umane de învățare, având mereu un substrat algoritmic care dorește a transpune ceea ce individul nu poate explica verbal, dar poate interpreta matematic.

Bibliografie

1. A.D.Dongare, R.R.Kharde, Amit D.Kachare, „Introduction to Artificial Neural Network”
<https://citeseerx.ist.psu.edu/document?rep1=rep1&type=pdf&doi=04d0b6952a4f0c7203577afc9476c2fcab2cba06>
2. Ethem Alpaydin, “Introduction to Machine Learning, Fourth Edition”
https://books.google.ro/books?hl=ro&lr=&id=uZnSDwAAQBAJ&oi=fnd&pg=PR7&dq=machine+learning+introduction&ots=xOuTrtlluV&sig=oEFcrLjdrFvhCb4c8SDD_TtKpQl&redir_esc=y#v=onepage&q&f=true
3. Grant Sanderson, „Neural Networks” <https://www.3blue1brown.com/topics/neural-networks>
4. Josh Starmer „K-nearest Neighbours”
<https://www.youtube.com/watch?v=HVXime0nQeI&t=1s>
5. Keith D. Foote, „A Brief History of Machine Learning”
<https://www.dataversity.net/a-brief-history-of-machine-learning/>
6. Marcin Andrychowicz, Misha Denil, Sergio Gómez Colmenarejo, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, Nando de Freitas, “Learning To Learn by Gradient Descent By Gradient Descent”
https://proceedings.neurips.cc/paper_files/paper/2016/file/fb87582825f9d28a8d42c5e5e5e8b23d-Paper.pdf
7. Zhongheng Zhang ” Introduction to machine learning: k-nearest neighbors”
<https://pmc.ncbi.nlm.nih.gov/articles/PMC4916348/>